Sentinel Logic

Collusion Detection and Response: Oversight That Outlives the Transcript[1]

Mikey Whiston
Security Researcher & Systems Architect
Sentinel Logic Ltd
mikey@mikeywhiston.dev

[1]Research conducted at the Swarm Alignment Hackathon, November 2026

Abstract

We recorded 42 file interactions between model-driven agents sharing one sandbox. A monitor reading only their declared transcript detected none of them, including the episodes in which every tool call was declared accurately and the binary answering to the declared name was hostile. Oversight of agent swarms reads a record the agent itself writes, whether that record is a transcript, a tool call, or a thinking trace. The failure is structural, so we build CDR, the oversight that does not read it. A kernel probe attributes every file touch to an agent by cgroup inode, an identifier the agent cannot choose, and emits an edge when two agents touch the same file. A policy then decides whether that edge was sanctioned, so the detector records what agents did without needing to infer why. Across 80 scripted test cases spanning 19 filesystem channels, we recover 88 of 88 interactions with zero false edges, and no episode in a benign control group of 40 produces an unsanctioned edge. Agents colocated in one container are separable when the harness partitions them into cgroups. The mechanism we use covers storage channels that the kernel reports as syscalls; timing channels, network egress and asynchronous I/O fall outside coverage.

1 Introduction

A modern agent with access to a shell is the most useful type of agent, but it is also the least observable one. It can write files, replace binaries, spawn processes, and manufacture its own evidence, so every record of what it did is self-reported rather than observed. That is not a hypothetical: we ran eighteen episodes of model-driven agents in a shared sandbox and the kernel observed 42 file interactions between them, of which a monitor reading only the declared tool calls detected none.

Existing oversight for agents reads exactly those records. Observability platforms log tool calls inside the runtime loop, harnesses score the transcript, and supervisors read what the agent says it is doing. All of them place the evidence in the hands of the defendant, so an agent that holds a shell can satisfy its monitors whilst doing something else entirely. The failure is sharper than ordinary lying, however. In our *laundering* episodes, the transcript is accurate because the agent ran the command it described, but the binary answering to that name had been replaced by an earlier step to a hostile one. The agent's declaration matches the execution, and the interaction remains invisible. Each of these systems produces telemetry about one subject. None of them reports what two agents did to each other.

We move the evidence out of the agent's reach. We call the resulting system CDR. We assume the agent has root inside its container but cannot change its cgroup membership, attach a probe, or modify the kernel. A kernel probe on the host records each file touch with the cgroup inode of the process that made it, resolves the object by device and inode inside the container's mount namespace, and emits an edge when two different agents touch the same object. An agent can rename itself, rewrite its transcript, or replace its own tooling, but it cannot alter its position in the cgroup hierarchy, the inode it touched, or the kernel's record of its syscalls. A policy file then decides whether an edge was sanctioned; the detector records all interactions and never infers the agent's intent.

On 80 scripted episodes over 19 filesystem channels, we recover 88 of 88 interactions with zero false edges, and 40 benign episodes produce no unsanctioned edges. On the live suite, the kernel records 42 interactions whereas a transcript monitor (representing a supervisor model) recovers none.

Our main contributions for this paper are:

1. **An agent identity and object model that cannot be altered by the subject.** Agent

identity is a cgroup inode and the object key is the kernel's (device, inode) pair, so coverage follows from the mechanism rather than a finite list of applicable channels. Syscalls are categorised into mutators and sensors, so a channel we never anticipated still works if the kernel reports it as a syscall naming an object.

2. **A measure of how much oversight transcript-only monitoring provides.** Across a scripted suite with pre-registered ground truth and a live suite of model-driven agents, transcript-based monitoring recovers none of the 42 observed interactions, while the kernel recovers all 88 of 88 in the scripted suite with no false edges.

3. **A verdict model that separates observability from interpretation.** Collusion is strictly defined as an unsanctioned edge, so the detector reports what agents did and the policy decides whether it was permitted, which keeps the mechanism auditable and its alarms revisable.

2 Related Work

3 Methods

4 Results

5 Discussion and Limitations

Limitations

Future Work

6 Conclusion

Code and Data

LLM Usage Statement